

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

June 24, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella hold della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare i requisiti forniti dalla committente in modo preciso e non ambiguo, costruire un primo modello logico dei macro-componenti del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o

2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

2.1. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta

il container dall'IOPort a slot5, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Risposta della committente La collocazione dell'IOPort nella griglia è da intendersi informalmente come mostrato nella figura presente nella traccia.

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è engaged viene semplicemente refusata / respinta con retrylater senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (retrylater o refused) senza code di attesa.

Risposta della committente Si conferma che le richieste non devono essere bufferizzate. Il sistema accetta le richieste mediante l'IOPort e, se questa risulta occupata, non deve essere possibile inviare altre richieste.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Risposta della committente Non è previsto lo svuotamento degli slot. Sarà l'obiettivo di un progetto futuro.

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato Out of service quando il sonar rileva una distanza $D > D_{FREE}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato Service working. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{FREE}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

Risposta della committente Si conferma che l'interpretazione è corretta.

3. Requirement analysis

3.1. Motivazione dell'uso del linguaggio QAK

QAK è il linguaggio messo a disposizione dalla nostra software house (unibo.issLab) per modellare sistemi software distribuiti, particolarmente espressivo nel formalizzare il concetto di **attore autonomo** e di **messaggio**, riducendo di molto l'“Abstraction Gap” tra requisiti nel contesto di un sistema distribuito eterogeneo. La natura **reattiva**

e **proattiva** di cargoservice, che deve rispondere a stimoli esterni e avviare autonomamente sequenze di azioni, è infatti catturata in modo naturale da un attore QAK, cosa che un POJO (Plain Old Java Object), componente passivo attivato da chiamate sincrone, non catturerebbe altrettanto bene. Dal modello QAK viene generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un **primo prototipo osservabile già nello Sprint 0**, prima ancora di scrivere una riga di logica applicativa.

3.2. Macro-componenti e natura software

3.2.1. cargoservice

Il cargoservice è l'orchestratore principale del sistema. Gestisce le richieste di carico, verifica le condizioni della hold, controlla i timeout e coordina le altre componenti.

Il componente è da sviluppare.

Dal punto di vista della natura software, presenta un comportamento sia reattivo (gestione di richieste ed eventi) sia proattivo (invio di comandi e aggiornamenti). Per questo motivo si ritiene opportuno rappresentarlo come un QAK Actor.

```
QActor cargoservice context ctxCargoService {  
    // DA SVILUPPARE: definizione degli stati e delle transizioni secondo  
    i requisiti  
}
```

3.2.2. cargorobot

Il **cargorobot** è l'entità incaricata di governare il DDR per la movimentazione dei container all'interno della hold.

Il componente è da sviluppare.

Dal punto di vista della natura software, la scelta architetturale è ancora oggetto di analisi. Se fosse modellato come un semplice POJO, il cargoservice gli cederebbe il controllo durante la movimentazione e non potrebbe reagire ad altri eventi del sistema fino al completamento dell'operazione. Se invece fosse realizzato come QActor, la comunicazione avverrebbe in modo asincrono, garantendo un maggiore disaccoppiamento tra le componenti.

3.2.3. IOPort

L'IOPort rappresenta l'interfaccia tra customer e sistema, composta da pushbutton e display.

Il software dell'interfaccia è da sviluppare.

Dal punto di vista della natura software, può essere realizzato come componente separato dal cargoservice.

```
QActor ioport context ctxCustomer {
    // DA SVILUPPARE
}
```

3.2.4. sonar

Il sonar rileva la presenza di un container nella sensor_area.

Il dispositivo fisico è considerato fornito ed è collegato al PicoW, mentre il software di integrazione è da sviluppare.

```
QActor sonar context ctxDevices {
    // DA SVILUPPARE
}
```

3.2.5. markerdevice

Il markerdevice etichetta i container depositati nello slot5 e notifica il completamento della marcatura.

Il dispositivo è considerato disponibile in forma simulata, mentre il software di controllo è da sviluppare.

```
QActor markerdevice context ctxDevices {
    // DA SVILUPPARE
}
```

3.2.6. LED

Il LED indica lo stato engaged/disengaged del sistema.

Dopo una consultazione con la committente, si è definito che il LED è un dispositivo fisico integrato all'interno del PicoW che gestisce il sonar. Il software di controllo è da sviluppare.

3.2.7. hold

La hold mantiene lo stato logico della stiva e l'occupazione degli slot.

Il componente è da sviluppare.

Dal punto di vista della natura software, può essere rappresentata come una semplice struttura dati passiva, responsabile della memorizzazione dello stato della stiva. Una possibile implementazione consiste in una matrice bidimensionale di celle, in cui ogni elemento rappresenta una posizione dell'ambiente e ne descrive il contenuto (area libera, ostacolo, slot, IOPort, Home, area sensore, ecc.).

```
enum CellType {
    FREE, OBSTACLE, HOME, SENSOR,
    IOPORT, SLOT1, SLOT2, SLOT3, SLOT4, SLOT5
}
```

```
}
```

```
CellType[][] hold = new CellType[6][7];
```

Legenda: H = HOME S = sonar/sensor area 1-4 = slot1-4
5 = slot5 I = IOPort X = ostacolo . = libero

```
col0 col1 col2 col3 col4 col5 col6
riga0 [ H ][ S ][ . ][ . ][ . ][ . ][ . ]
riga1 [ . ][ 1 ][ X ][ X ][ 2 ][ . ][ . ]
riga2 [ . ][ . ][ . ][ . ][ X ][ 5 ][ . ]
riga3 [ . ][ 3 ][ X ][ X ][ 4 ][ . ][ . ]
riga4 [ . ][ . ][ . ][ . ][ . ][ . ][ . ]
riga5 [ I ][ . ][ . ][ . ][ . ][ . ][ . ]
```

3.3. Formalizzazione dei messaggi QAK

Dai requisiti, l'unica richiesta che si evince è quella di carico, che viene modellata come request in qak. La richiesta viene inviata dal customer al cargoservice tramite l'IOPort.

```
Request load_request : loadRequest(none)
Reply load_accepted : loadAccepted(slotID) for load_request
Reply load_retrylater : loadRetryLater(none) for load_request
Reply load_refused : loadRefused(none) for load_request
```

3.4. Contesti logici

CONTESTO	COMPONENTI E RESPONSABILITÀ
ctxCargoService	Contiene l'attore cargoservice. Nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico.
ctxCustomer	Raggruppa le entità dedicate all'interazione con l'utente: IO-Port (con display e pushbutton) e LED.
ctxDevices	Raggruppa i dispositivi presenti nella hold: sonar, hold e markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.5. Core business

Il **core business** del sistema è la gestione del ciclo di carico di un container. La responsabilità della sequenza applicativa resta in **cargoservice**: il cargorobot è coinvolto per eseguire spostamenti richiesti dal servizio, non per decidere se una richiesta debba essere accettata, rifiutata o sospesa. La sequenza principale ricavata dai requisiti è espressa dal metamodello come segue:

System cargosystem

// Vocabolario delle interazioni

Request load_request : loadRequest(none)

Reply load_accepted : loadAccepted(slotID) for load_request

Reply load_retrylater : loadRetryLater(none) for load_request

Reply load_refused : loadRefused(none) for load_request

Event sonar_distance : distance(D)

Dispatch marking_done : markingDone(containerID)

Context ctxcargoservice ip [host="localhost" port=8050]

// Core Business

QActor cargoservice context ctxcargoservice {

State s0 initial {

println("cargoservice | started")

}

Goto disengaged

State disengaged {

println("cargoservice | DISENGAGED: waiting for load_request...")

}

Transition t0

whenRequest load_request -> handle_load_request

State handle_load_request {

printCurrentMessage

// Simulazione accettazione

replyTo load_request with load_accepted : loadAccepted(1)

}

Goto engaged

State engaged {

println("cargoservice | ENGAGED: waiting for container (sonar)...")

}

Transition t1

whenTime 10000 -> handle_timeout // Requisito: timeout

deposito

whenEvent sonar_distance -> move_to_slot5 // Requisito: rilevamento

container

State handle_timeout {

println("cargoservice | TIMEOUT: container non depositato, libero

l'IOPort")

}

Goto disengaged

State move_to_slot5 {

println("cargoservice | CONTAINER RILEVATO: avvio movimentazione

verso slot 5")

// Qui ci sarà la delega al cargorobot

}

```

    Goto disengaged
}

```

4. Test plan

I test funzionali verificano il comportamento osservabile del sistema rispetto ai requisiti, indipendentemente dall'implementazione.

SCENARIO	PRECONDIZIONI	RISULTATO ATTESO
Accettazione	disengaged , non OoS, IOPort libera, slot disponibile	load_accepted con slot riservato; stato engaged ; LED lampeggiante
Rifiuto (hold piena)	disengaged , non OoS, IOPort libera, slot1-4 tutti occupati	load_refused ; stato disengaged ; LED spento
Retrylater (OoS)	Sistema Out of service	load_retrylater ; stato immutato
Retrylater (IOPort occupata)	disengaged , non OoS, IOPort occupata da un container	load_retrylater ; stato disengaged
Timeout deposito	Sistema engaged ; customer non deposita entro il tempo prefissato	Sistema disengaged ; LED spento
Ciclo completo	disengaged , non OoS, IOPort libera, slot disponibile	Container nello slot riservato; display “ Service working ”; disengaged ; LED spento
Guasto sonar	Sistema operativo; sonar misura $D > D_{FREE}$ per ≥ 3 s	Sistema Out of service ; display “ Out of service ”
Rilevazione container	Sistema engaged ; sensor area vuota; sonar misura $D < D_{FREE}/2$ per ≥ 3 s	cargoservice avvia la movimentazione verso slot5

5. Project

Dall'analisi dei requisiti si propone, come ipotesi iniziale, una struttura a **tre sprint** con integrazione progressiva dei componenti. La suddivisione serve a organizzare il lavoro e i test, non a fissare decisioni architetturali definitive.

5.1. Sprint 1: Core business

Goal: realizzare il primo nucleo eseguibile di cargoservice con collaboratori simulati.

Funzionalità: ciclo di carico completo con collaboratori simulati, modello dello stato della hold, LED simulato, timeout.

5.2. Sprint 2: IOPort, display e sonar

Goal: rendere esplicita l'interazione con IOPort, display e sonar a livello software.

Funzionalità: IOPort come interfaccia web, sonar simulato (rilevazione container e malfunzionamento), aggiornamento display con stato e messaggi.

5.3. Sprint 3: Dispositivi fisici

Goal: integrare, dove disponibili, i dispositivi fisici o i servizi forniti e verificare il comportamento end-to-end.

Funzionalità: cargorobot e servizio DDR disponibile, sonar fisico, marker device fisico, LED fisico.

6. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340